# DTC: Addressing the long-tailed problem in intrusion detection through the divide-then-conquer paradigm

Chaoqun Guo, Nan Wang*, Yuanlin Sun, Dalin Zhang
*dept. School of Software Engineering*
*Beijing Jiaotong University*
Beijing, China
{chqguo, wangnanbjtu, 23126470, dalin}@bjtu.edu.cn

***Abstract*—Intrusion detection systems (IDS) analyze the monitored data to detect patterns or signatures that correspond to known cyberattack techniques, vulnerabilities, or deviations from established baselines. They employ different algorithms and techniques to identify potential threats. When building "Known Patterns and Signatures", it always faces the long-tailed problem, which refers to the imbalanced distribution of different types of network traffic or events in a dataset, which means that there are very few instances of certain types of intrusions compared to more common types of network traffic or benign events. Such a situation poses a great challenge to deep learning-based or machine learning-based detection models on how to handle it. Models may struggle to learn from long-tailed distributions because they tend to bias their predictions toward the majority, and perform well on common events but poorly on rare intrusions. To address this problem, different from previous methods concerning training the model on the whole samples to obtain a balanced data distribution, we first focus on dividing the whole samples into a balanced group and an imbalanced one, then, we train a detection model on the balanced group. Cycle over and over again. Specifically, We propose to use a Gaussian mixture flow filter to progressively perform sample aggregation, continuously transforming the long-tail distribution into a more balanced which allows us to train the classifier on the obtained balanced group. The separated training samples with high distribution balance make it easier to train subsequent classifiers and mitigate the head-to-tail bias. Through extensive experiments, we have achieved new state-of-the-art performance on common intrusion detection datasets such as UNSW-NB15, CIC-IDS2017, and NSL-KDD. These results demonstrate that it is possible to surpass carefully constructed balanced datasets by progressively distinguishing the head class and the tail class.**

***Index Terms*—Intrusion detection, Long-Tailed learning, Normalizing flows, Gaussian Mixture Model.**

## I. Introduction

Intrusion detection[1] is a fundamental component of modern layered cybersecurity strategy. It is an ongoing process of monitoring a computer system or network for unauthorized access or malicious activities by analyzing the monitored data for patterns or signatures, which match known cyberattack techniques, vulnerabilities, or deviations from established baselines and taking action to prevent or mitigate potential threats.

*Nan Wang is the corresponding author, email: wangnanbjtu@bjtu.edu.cn

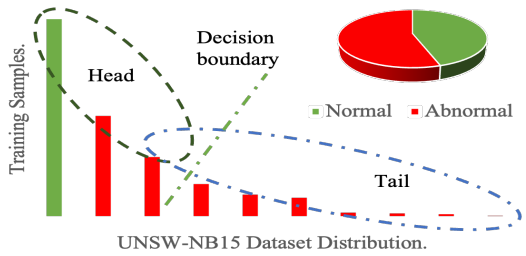


Fig. 1. Illustration of UNSW-NB15 dataset distribution. The classifier decision boundary does not lie in the low-density region in the data space.

Research related to intrusion detection[2, 3] has been developed rapidly in the past decade, driven by the development of machine learning, deep learning and large-scale intrusion datasets[4, 5]. Previous works focused on training detection models on artificially constructed balanced datasets to build "Known Patterns and Signatures". But the real situation is not the same, intrusion detection often faces the long-tail problem, see Figure. 1. The distribution of different types in the dataset is imbalanced, which is reflected in the more common types of network traffic or benign events, while there are very few instances of certain types of intrusions. Previous learned detection models dominated by head classes tend to perform better on head classes, while performance is significantly worse for tail classes with limited samples. Therefore, models learned through common practices of empirical risk minimization[6] cannot handle practical applications where long-tailed distributions occur in intrusion detection. Current methods, like random oversampling, enlarge the dataset by duplicating a limited number of samples, leading to heightened model training complexity and overfitting issues. On the other hand, undersampling might result in information loss, leading to underfitting of the model. Both of these approaches can alter

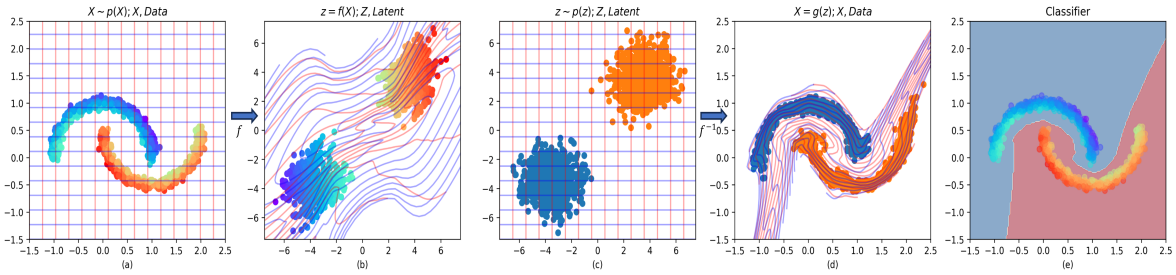


Fig. 2. Illustration of Gaussian mixture flow model based on normalizing flow on a balanced binary classification problem. Colors represent the two classes or the corresponding Gaussian mixture components. (a): Original Data distribution. (b): The learned mappings of the data to the latent space. (c): Samples from the Gaussian mixture in the latent space. (d): Samples from the model in the data space. (e): The classifier decision boundary lies in the low-density region in the data space. The transformation map is denoted as $f$, and the inverse transformation map is denoted as $f^{-1}$.

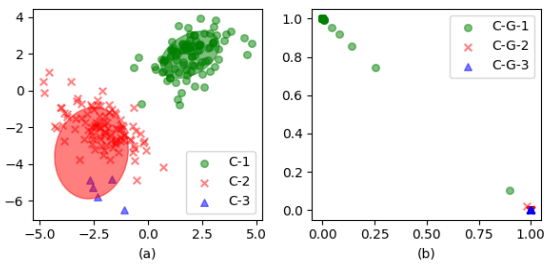


Fig. 3. Illustration of Gaussian mixture model on a long-tailed detection problem. (a): The colored points represent different classes (C-1, C-2, C-3), where the blue △ are tails, and the red × and green ○ are heads. The green and red ellipses are two Gaussian components constructed to model these three classes. The points in the green ellipse only contain green ○, and the data purity in this Gaussian component is relatively high. The points in the red ellipse contain red × and blue △, and the data purity is relatively low, and the next separation is required. (b): The colored points represent the latent space learned by the Gaussian mixture model (C-G-1, C-G-2, C-G-3). To facilitate understanding, we use the same colors and legend as in (a) for representation, and the corresponding color and legend points are the same class of data. The red × and blue △ are classified into the same category by the Gaussian mixture model.

the original data distribution, introducing bias. Therefore, it's particularly crucial to establish a balanced data distribution that aligns with the original data distribution.

To address this problem and improve the detection performance of the classifier on all classes, we suggest addressing the head-to-tail bias bottleneck and tackling long-tail intrusion detection through a **divide-then-conquer** paradigm (DTC). We propose to progressively aggregate the label space, and continuously aggregate the original long-tail distributed labels into a balanced space so that it can separate the head and tail classes. Specifically, see Figure. 3, we use Flow Gaussian mixture filter based on normalizing flow to progressively transform the label space of the long-tail distribution in the original data into a balanced distribution through a series of reversible neural networks, and separate out the label space in the new space. The separated head class has a high data balance, then we train a classifier on such well-separated samples. The subsequent tail classes can continue to aggregate labels according to the training situation of the model to achieve a new balance space.

In implementation, we have the flexibility to link multiple customized modules according to the task requirements. These modules can be cascaded throughout the entire training process, adapting to the progress of training. This approach allows us to address imbalance bias gradually by systematically separating head and tail classes. These customized modules can serve specific functions, such as data augmentation and transfer learning, as needed.

The key contributions of this work can be summarized as follows:

- We first propose to deal with the long-tailed intrusion detection with normalizing flow in a divide-then-conquer paradigm.
- We propose to progressively update the separation threshold to obtain higher purity heads and tails.
- We conduct extensive experiments and set new state-of-the-art performance on common intrusion detection datasets.

## II. Related Work

### A. Normalizing Flow

Normalizing flow was defined by Tabak[7] in "Density estimation by the dual ascent of the log-likelihood", while Tabak and Turner[8] provided a general mechanism for defining expressive probability distributions, only requiring the specification of a base distribution and a series of bijective transformations. With tractable distributions which be able to efficiently and accurately used for sampling, density evaluation[9], **clustering and classification**[10].

Transforming a simple probability distribution, such as a standard normal distribution $\mathcal{X}$, into a more intricate distribution $\mathcal{Z}$ through a series of invertible and differentiable mappings $f : \mathcal{X} \to \mathcal{Z}$. It can be seamlessly employed in the realms of clustering and classification. We briefly illustrate how it works in Figure. 2, (we referred some codes from literature[11]). Training such a network $g$ that maps latent variable $z \sim p_{\mathcal{Z}}$ to a sample distribution $x \sim p_{\mathcal{X}}$, if $g$ is

bijective, it can be trained through maximum likelihood using the *change of variable formula*:

$$p_{\mathcal{X}}(x) = p_{\mathcal{Z}}(z) \cdot \left| det(\frac{\partial g(z)}{\partial z^T}) \right|^{-1}. \tag{1}$$

Given an observed data variable $x \in \mathcal{X}$, a simple prior probability distribution $p_{\mathcal{Z}}$ on a latent variable $z \in \mathcal{Z}$, and a bijection $f^{-1} : \mathcal{Z} \to \mathcal{X}$ (with $g = f^{-1}$), the change of variable formula defines a model distribution on $\mathcal{X}$ by

$$p_{\mathcal{X}}(x) = p_{\mathcal{Z}}(f(x)) \cdot \left| det(\frac{\partial f(x)}{\partial x^T}) \right|, \tag{2}$$

$$log(p_{\mathcal{X}}(x)) = log(p_{\mathcal{Z}}(f(x))) + log(\left| det(\frac{\partial f(x)}{\partial x^T}) \right|), \tag{3}$$

where $\frac{\partial f(x)}{\partial x^T}$ is the $Jacobian$ of $f$ at $x$.

The volume change is the product of the absolute values of the determinants of the $Jacobians$ for each transformation, as required by the change of variables formula. This approach yields a mechanism for generating novel distribution families. It involves selecting an initial density and subsequently stringing together a series of parameterized, invertible, and differentiable transformations. The resulting density can be generated via sampling (by initially sampling from the chosen density and then applying the transformations), and the density at a sampled point.

Some approaches based on normalizing flow pass the class labels as an input to coupling layers, conditioning the output of the flow on the class, such as RealNVP[12], have used class-conditional sampling where the transformations are conditioned on the class. Deep Invertible Generalized Linear Model[13] trains a classifier on the latent representation of a normalizing flow to perform supervised or semi-supervised classification. Our DTC also builds on it, but we do not train a classifier on the latent representation, we use it to progressively filter out head and tail samples, then train the classifier on the filtered samples.

### B. Long-tailed Problem

Long-tailed learning seeks to learn a model from a large-scale dataset with a long-tailed class distribution[14], faces two main difficulties: 1) imbalanced data numbers across classes make models biased to head classes and performance poorly on the tail; 2) lack of tail-class samples makes it further challenging to train models for tail-class classification. Such a task is fundamental and may occur in various recognition tasks, such as network traffic classification[15], intrusion detection[4].

To address this problem, massive studies[16, 17] have been conducted in recent years. These existing studies can be grouped into three main categories based on their main technical contributions, i.e., class re-balancing, information augmentation and module improvement. **Class re-balancing** seeks to re-balance the negative influence brought by the class imbalance in training sample numbers, such as re-sampling which refers to the process of adjusting the distribution of a dataset by either adding more instances or removing some instances from certain classes or groups within the data. **Information augmentation** seeks to introduce additional information into the training process, such as transfer learning which transfers the knowledge from a source domain to enhance model training on a target domain, e.g., from network traffic classification to intrusion detection. **Module improvement** seeks to improve network modules through different strategies, such as representation learning which focuses on automatically learning and extracting meaningful and informative representations or features from given data. Classifier design aims to design a classifier, e.g., Linear Classifier, Realistic Taxonomic Classifier suitable for a specific task by using different strategies.

## III. Divide Then Conquer

### A. Problem definition

A probability distribution is considered long-tailed if its tail behavior or tail decay rate is slower than that of a comparable exponential distribution. For some positive constants $C$ and $\alpha$, the tail probability of the distribution, denoted as $P(X > x)$ for large values of $x$, follows a power-law decay: $P(X > x) \sim C * x^{-\alpha}$. We define the notation used in our work. Let $X = \{x_i, y_i\}, i \in \{1, ..., n\}$ be a training set, where $y_i$ is the label for a data point $x_i$. Let $n_j$ denote the number of training samples for class $j$, and $n = \Sigma_{j=1}^{C} n_j$ be the total number of training samples. Without loss of generality, we assume that the classes are sorted by cardinality in decreasing order, i.e., if $i < j$, then $n_i \geq n_j$. Additionally, since we are in a long-tailed intrusion detection setting, $n_1 \gg n_j$. The imbalance of data is measured by the imbalance factor $\beta = n_1/n_j$, and the $\beta$ of a typical long-tailed problem ranges from 10 to 1000.

Previous work[18] has shown that data imbalance mainly affects classifiers and has little impact on feature learning, moreover, instance-balanced sampling learns the best and most generalizable representations; at the same time, some self-supervised contrastive learning methods[19] can balance feature learning by introducing instance-level transformation invariance that does not involve labels. Combining these two aspects and incorporating instance-level multi-view invariance and class contrast, supervised contrast learning[20] achieve even more outstanding results. Since the head-tail separation can help long-tailed learning, the main work of this paper is to separate head and tail samples progressively and classify them in a divide-then-conquer manner. We re-define a more balanced label space $h \in \{1, 2, ..., K\}$ to divide the heads and tails. The higher the degree of separation between the heads and tails, i.e., map the data and labels to a balanced space, the easier it is to learn detection models. Achieving an balanced space is always impractical, we propose to progressively map and cluster samples, and filter out the samples that are well subject to the balanced distributions, so we force the model to learn head-tail separability during the clustering, and transform

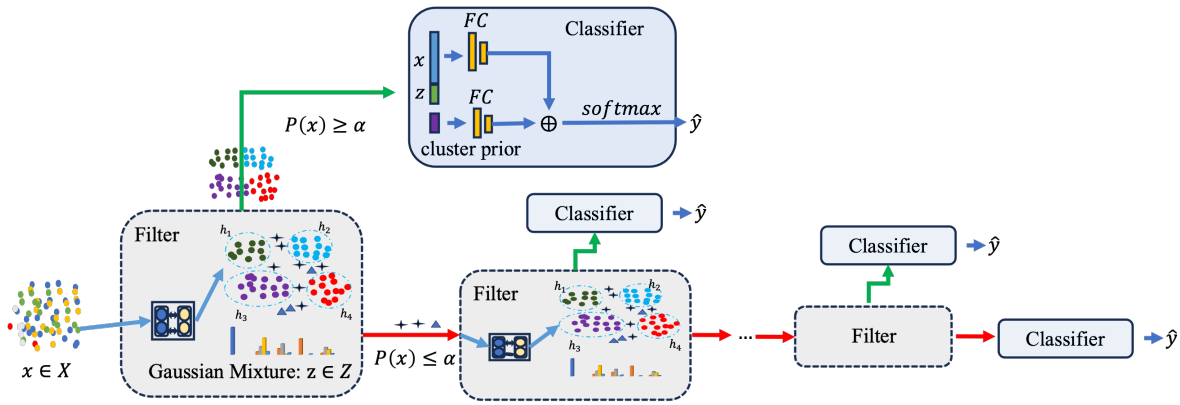


Fig. 4. Overview of DTC. Gaussian mixture flow filter clusters the data $z$ in latent space and assigns new cluster labels $H$. Then, the filter divides the samples into two groups: the well-clustered and the rest outliers. The Cluster-aided Classifier exploits the cluster information in detection. The well-clustered samples are filtered out and sent to the classifier, while the remaining outliers are either forwarded to the next flow filter or sent to the final classifier.

the long-tailed learning to the label space learning and latent space mapping progress, while these can be more easily settled with the help of the inductive bias and separated samples and the transformation ability of normalizing flow. As mentioned in related work, we utilize normalizing flow to simultaneously map the data $x$ to representation $z$ in a more balanced latent space and estimate the new class label $h$. There are multiple constraints for the normalizing flow to re-balance the latent space, our code is built on the top of literature[21].

### B. Overview

The pipeline of our method is depicted in Figure. 4, there are mainly two types of modules arranged in a cascaded manner like DLSA[21]. Gaussian mixture flow filter clusters the data $z$ in latent space and assigns new cluster labels $H$. Then, the filter divides the samples into two groups: the well-clustered and the rest outliers. The Cluster-aided Classifier exploits the cluster information in detection. The well-clustered samples are filtered out and sent to the classifier, while the remaining outliers are either forwarded to the next flow filter or sent to the final classifier.

*1) Gaussian mixture flow filter:* The Gaussian mixture flow filter module incorporates a normalizing flow model $f_\theta^{-1}: \mathcal{X} \rightarrow \mathcal{Z}$ with trainable parameter $\theta$, mapping input data sample $x \in \mathcal{X}$ to latent distribution $z \in \mathcal{Z}$. We assume the $P(z)$ is a Gaussian mixture and each component corresponds to a cluster, then, we can cluster the input data into $K$ clusters with mean $\mu_k$ and covariance $\Sigma_k$:

$$P_{\mathcal{Z}}(z|h=k) = \mathcal{N}(z|\mu_k, \Sigma_k), \tag{4}$$

$$P(z) = \sum_{k=1}^{K} P(h=k)P(z|h=k), \tag{5}$$

With the balance assumptions, prior probability $P(h)$ which determines the size of a Gaussian component, been set to a uniform distribution, can be a tractable optimization method of normalizing flow with the Gaussian mixture model, thus:

$$P(z) = \frac{1}{K}\sum_{k=1}^{K} P(z|h=k), \tag{6}$$

and the prediction of sample $x$ is given by $Bayes'$ theorem:

$$P(k|x) = \frac{P(z|h=k)}{\sum_{k'=1}^{K} P(z|h=k')}, \ \hat{h} = arg\max_k P(k|x). \tag{7}$$

We train the filter model with end-to-end gradient descent optimization with three objectives: 1) improved maximum likelihood loss and head-tail separability, 2) cluster balance Loss, 3) purity loss. These three losses exists in DLSA. With the elegant invertible property of normalizing flow and change of variable formula Equation. 2, we can obtain the likelihood of a sample $x$:

$$\mathcal{L}(\theta; x) = \sum_{k=1}^{K} \frac{1}{K}\mathcal{N}(f(x)|\mu_k, \Sigma_k) \cdot \left| det(\frac{\partial f(x)}{\partial x^T}) \right|. \tag{8}$$

To enhance the separability of head and tail classes of the filter, DLSA incorporates *sample weighting* to impose higher weights on tail samples than head by:

$$\omega(i) = \frac{n_i^{-q}}{\Sigma_j n_j^{-q}}, \tag{9}$$

where $n_i$ is the sample size of class $i$, $q$ is a positive number ranging from 1 to 2, to make the tail samples receive larger weights, then, weighted maximum likelihood loss is constructed with the negative log-likelihood of a data batch $\mathcal{B}$:

$$\mathcal{L}_{mle}(\mathcal{B}) = -\sum_{x \in \mathcal{B}} \omega(y) log\mathcal{L}(\theta; x). \tag{10}$$

This simple weighting method is effective to some extent, as it takes advantage of the fact that the distribution of data features makes it somewhat easier to learn characteristics of the head classes compared to the tail classes. By assigning greater weights to the tail data, it biases the model toward learning features of the tail classes. Sihao and Jiafeng[22] observe that even the head classes contain difficult instances

to learn, by reducing the weights of these head classes, such instances would become more difficult to learn and hurt the performance consequently, so they propose to dynamically adjust the sampling probabilities of instances according to the instance difficulty. Then the improved maximum likelihood loss be updated as:

$$\mathcal{L}_{dmle}(\mathcal{B}) = -\sum_{x \in \mathcal{B}} \omega_{x,T}(y) log \mathcal{L}(\theta; x), \tag{11}$$

where $\omega_{x,T}$ is the estimated difficulty after $T$ iterations for a given batch instances $x$.

To avoid model collapse and degeneration where $f^{-1}$ may map all training sample to one component and enhance purity in each component, We retained the calculation formulas for cluster balanced loss $\mathcal{L}_{cbl}$ and purity loss $\mathcal{L}_{pl}$ in DLSA (see Equation. 8, 9, 10 in [21]).

Then, the total loss is:

$$\mathcal{L}_{total} = \lambda_{dmle}\mathcal{L}_{dmle} + \lambda_{cbl}\mathcal{L}_{cbl} + \lambda_{pl}\mathcal{L}_{pl}. \tag{12}$$

After filter been trained, we use a dynamic likelihood threshold $\alpha$ to separate the well-clustered samples and outliers:

$$stage_threshold = 1 - rec_depth/max_depth, \tag{13}$$

$$\alpha = max(stage_threshold, llh_proportion). \tag{14}$$

Where $rec_depth$ and $max_depth$ represent the current recursion depth and the total recursion depth. When dealing with the well-clustered majority, it is advisable to select a smaller threshold which helps to mitigate the impact of noise generated by these samples. On the other hand, for the minority samples, opting for a larger threshold can lead to the development of a more robust classifier.

*2) Classifier:* The well-clustered samples filtered out by the model are sent to a Cluster-aided Classifier, we employ a Near-Class-Mean[23](NCM), which assigns a data point to a class based on its proximity or similarity to the mean feature vector of that class, to model these samples. During the training phase, NCM calculates the mean feature vector (mean value of each feature) for each class in the training dataset. These mean vectors represent the "center" of each class in the feature space, it takes advantage of the consistency of the data distribution on each Gaussian component. The best situation is that each component $h$ contains only one class $y$ so that the prediction results can be obtained by simply mapping from cluster $h$ to label $y$. The real situation is that the obtained clusters $H$ are noisy and each cluster $h$ contains samples from more than one class. We propose a label-class mapping approach where, for each sample $x$, we initially calculate the cluster $P(y|h = \hat{k}_i)$, representing the class frequency of training samples belonging to the predicted cluster $\hat{k}_i$. Subsequently, we pass the cluster prior vector and the concatenation of feature $x_i$ and latent variable $z_i$ through two separate fully connected layers ($FC$). The outputs of these layers are added together, and the result is passed through a $Softmax$ function to obtain the output probabilities. During classifier training, we employ instance-balanced sampling, as described in [18].

$$P(y|x_i, z_i) = Softmax(FC\left[x_i, z_i\right] + FC\left[P(y|h = k_i)\right]) \tag{15}$$

## IV. Experiments

### A. Experimental setup

*1) Datasets:* we perform extensive experiments on three large-scale intrusion detection datasets, including UNSW-NB15[24], CIC-IDS2017[25], and NSL-KDD[26]. UNSW-NB15 contains both normal network traffic and various types of network attacks or intrusions. It includes a wide range of attack categories, such as Denial of Service (DoS), Remote to Local (R2L), User to Root (U2R), and Probe attacks, among others with an amount from 37000 to 44. CIC-IDS2017 has 15 classes and the number of per class ranges from 2271320 to 11. NSL-KDD is an improved and extended version of the earlier KDD Cup 1999 dataset. It's **important to highlight that** we did not train the model on the original training dataset. The original data distribution did not align well with our experimental needs since it couldn't dynamically evaluate our model's performance under various imbalance rates. Instead, we employed a long-tail distribution function to randomly sample training data for our experiments from the original training set. This involved constructing multiple subsets of the original training set and conducting performance tests on all test data.

*2) Evaluation protocol:* after training on the long-tailed intrusion detection datasets, we evaluate the models on the corresponding balanced test dataset and report the commonly used mean accuracy over all classes. We also use Matthews correlation coefficient[27] (MCC) and normalized mutual information[28] (NMI) to measure the performance like DLSA.

*3) Implementation details:* in the divide stage, recognizing the intricacy of the distribution of high-dimensional data features and the limited modeling capability of shallow normalizing flows, we opt for a 5-layer masked autoregressive flow[29] as a flow filter model to learn the mapped latent space for UNSW-NB15, 2-layer for CIC-IDS2017. To assess the intrinsic efficacy of the model's initial architecture, we deliberately refrained from incorporating additional strategies that could potentially enhance long-tail classification, such as information augmentation and module improvements. Moving on to the conquer stage, we employ the Near-Class-Mean (NCM) classifier. This classifier assigns an input data point to the class whose mean (average) feature vector is most proximate to the feature vector of the input data point with an instance-level sampling strategy.

We compared it with the current mainstream methods for dealing with long-tail problems. These methods were chosen from three categories in related work: Class Re-balancing: we employ the Synthetic Minority Over-sampling Technique[30] (SMOTE) to balance the class distribution. SMOTE generates synthetic samples for the minority class. Additionally, the classifier selected for this is a linear classifier. Information Augmentation: we adopt the integration method within the Majority-to-Minority strategy. This includes the use of Balanced Random Forest and other techniques. Module Improve-

TABLE I

ACCURACY(ACC), MCC, NMI % ON UNSW-NB15, CIC-IDS2017 AND NSL-KDD WITH DIFFERENT IMBALANCE RATIO: $\beta$.

*NOTATION! LC: LINEAR CLASSIFIER, $\tau$: $\tau$ NORMALIZED CLASSIFIER, SMOTE: SYNTHETIC MINORITY OVER-SAMPLING TECHNIQUE, M2M: MAJORITY-TO-MINORITY.

(THE MAXIMUM VALUE OF THE INDICATOR FOR EACH SET OF EXPERIMENTS IS DISPLAYED IN **BOLD**.)

| Dataset Name | | | UNSW-NB15 | | | | CIC-IDS2017 | | | | NSL-KDD | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $n_1$, Classes | | | 3000, 10 | | | | 3000, 15 | | | | 3000, 5 | | | |
| Imbalance Ratio: $\beta = \frac{n_1}{n_c}$ | | | 10 | 100 | 200 | 500 | 10 | 100 | 200 | 500 | 10 | 100 | 200 | 500 |
| Methods | SMOTE | ACC | 68.1 | 65.2 | 65.1 | 62.9 | 80.5 | 64.1 | 63.2 | 51.3 | 70.6 | 67.4 | 66.7 | 59.2 |
| | | MCC | 61.4 | 56.1 | 55.6 | 53.5 | 74.6 | 45.6 | 42.6 | 20.6 | 59.7 | 57.2 | 56.1 | 51.7 |
| | | NMI | 52.4 | 49.9 | 49.3 | 47.6 | 67.0 | 40.9 | 37.6 | 24.1 | 53.3 | 52.7 | 51.7 | 68.35 |
| | M2M | ACC | 67.4 | 66.8 | 66.8 | 65.5 | 81.2 | 65.8 | 64.3 | 52.2 | 69.9 | 67.3 | 66.4 | 57.3 |
| | | MCC | 61.1 | **60.3** | **60.3** | **58.9** | 75.3 | 53.1 | 45.7 | 23.5 | 59.1 | 57.1 | 55.8 | 49.5 |
| | | NMI | **53.6** | 52.2 | **52.5** | 51.2 | 68.8 | 54.7 | 41.2 | 25.6 | **54.2** | 52.9 | 51.2 | 46.6 |
| | LC | ACC | 70.1 | 66.6 | 65.8 | 62.9 | 79.1 | 63.6 | 62.7 | 50.1 | 69.7 | 66.8 | 65.3 | 58.5 |
| | | MCC | **63.1** | 57.3 | 56.6 | 53.4 | 72.8 | 46.1 | 46.8 | 8.9 | 58.8 | 56.2 | 55.8 | 50.4 |
| | | NMI | 51.9 | 48.1 | 48.8 | 44.6 | 64.0 | 41.5 | 49.4 | 12.9 | 54.0 | 51.2 | 50.7 | 47.5 |
| | $\tau$ | ACC | 70.1 | 67.7 | 67.0 | 66.4 | 80.3 | 65.8 | 65.2 | 58.5 | 71.2 | 68.9 | 67.4 | 60.2 |
| | | MCC | **63.1** | 58.8 | 58.0 | 57.3 | 74.3 | 53.1 | 51.4 | 37.6 | 61.3 | 58.2 | 56.8 | 51.7 |
| | | NMI | 51.7 | 51.3 | 50.9 | 49.8 | 65.7 | 55.2 | 55.4 | 39.8 | 53.6 | 53.1 | 52.1 | 49.5 |
| | **DTC** | ACC | **71.2** | **69.3** | **68.1** | **66.9** | **95.4** | **94.8** | **93.6** | **90.4** | **72.8** | **69.4** | **68.2** | **61.5** |
| | | MCC | 62.2 | 58.5 | 57.4 | 57.5 | **92.7** | **91.9** | **88.7** | **83.7** | **63.2** | **58.6** | **57.6** | 53.3 |
| | | NMI | 53.1 | **52.7** | 52.1 | **51.9** | **87.1** | **85.9** | **82.1** | **73.6** | 54.1 | **53.8** | **53.2** | **49.1** |

ment: our focus is on enhancing classifier design. We opt for linear classifiers and tau normalized classifiers[14]. This choice reflects the inherent filtering performance of our flow filter model.

### B. Results and analysis

We conducted experiments on the above three datasets, under different balancing rates (from 500 to 10), and after comparing them with existing methods that can handle the long-tail problem (we have mentioned it in related work), we found that the DTC we proposed can effectively improve the accuracy of intrusion detection in the long-tail state.

*1) Results and comparison with existing methods:* we report the results in Table. I. On UNSW-NB15, compared with the method that obtains the average results, we obtain 1-2% gains in accuracy for each imbalance ratio. This improvement is reflected in various imbalance rates due to the excellent architecture of our model. Such a divide-then-conquer paradigm can effectively improve intrusion detection in long-tailed situations. On CIC-IDS2017, we achieved qualitative improvements, far ahead of other methods used in this work. When the imbalance rate is 100 and 200, the results of each method are not very different. After careful analysis, we found that when constructing these two types of data, due to the relatively low imbalance rate, the actual sampling exists a deviation between the number of samples used for training and the expected number. Since the actual number of samples of the corresponding class is smaller than expected, the sample spaces collected under the two uneven rates are almost the same, and the results obtained are also almost the same. On the other hand, the methods we have constructed are very robust. As the imbalance rate increases from 10 to 3000, the data distribution imbalance gradually intensifies, and the learning process becomes more difficult, which is reflected in the corresponding results gradually decreasing, which is consistent with common sense, see Figure. 6.

TABLE II

ABLATION STUDIES ON CIC-IDS2017. BASE MODEL: WITH 2 NORMALIZING FLOW FILTER BLOCKS, 10 GAUSSIAN COMPONENTS, MAX RECURSION DEPTH IS 3, $n_0 = 3000$ AND IMBALANCE RATIO IS 500.

| | | ACC | MCC | NMI |
|---|---|---|---|---|
| Baseline | | 90.4 | 83.7 | 73.6 |
| Loss importance | w/o $\mathcal{L}_{dmle}$ | 88.3 | 82.5 | 73.6 |
| | w/o $\mathcal{L}_{cbl}$ | 87.5 | 81.4 | 71.9 |
| | w/o $\mathcal{L}_{pl}$ | 86.3 | 80.2 | 70.1 |
| Gaussian components | 8 | 89.1 | 81.2 | 71.8 |
| | 11 | 90.1 | 83.1 | 73.5 |
| Filter architecture | 3 | 88.8 | 79.9 | 71.4 |
| | 6 | 92.9 | 88.3 | 81.2 |
| Recursion depth | 2 | 93.9 | 88.3 | 81.2 |
| | 4 | 95.1 | 91.7 | 85.8 |
| Scalability | with SMOTE | 90.6 | 83.9 | 74.0 |
| Extreme imbalance rate | 1000 | 89.8 | 82.7 | 73.7 |
| | 3000 | 82.4 | 65.7 | 64.8 |

*2) Ablation study:* in this subsection, we provide six ablation studies on CIC-IDS2017 with 3000 ($n_0$) and $\beta = 500$ (imbalance ratio). The results are reported in Table. II and Figure. 5. 1) **Loss importance**: we remove one of the three losses in turn to evaluate the impact of the corresponding loss on the model. The results show that removing any loss has an important impact on the model, and the loss setting we propose is necessary. The results of the loss impact verification demonstrate that when applying normalizing flow for filtering, prioritizing higher purity in the filtered Gaussian mixture components proves advantageous for enhancing model performance. 2) **Gaussian mixture components**: too many Gaussian mixture components will make the latent space

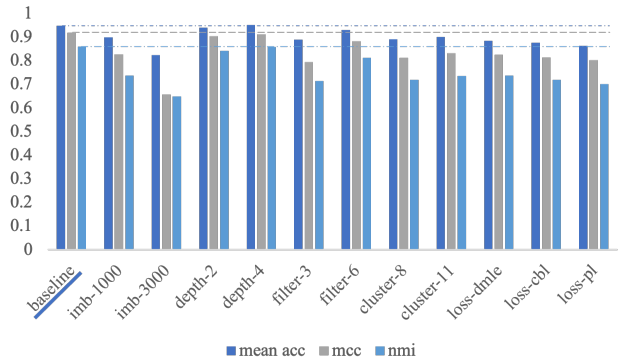


Fig. 5. Results of ablation Study on CIC-IDS2017.

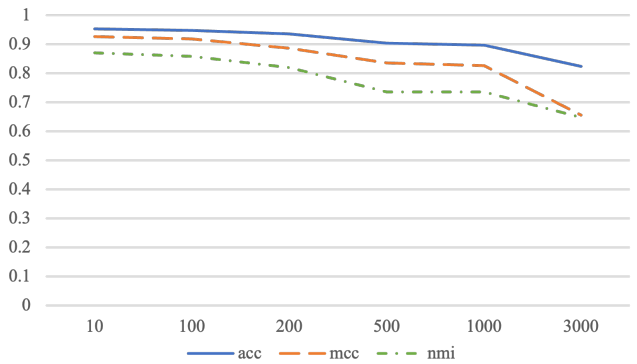


Fig. 6. Results of different imbalanced ratio $\beta$ on CIC-IDS2017.

sparse, without alleviating the long-tail distribution, while too few will result in low purity on each Gaussian component, which is not conducive to the separation of head and tail classes. 3) **Filter architecture**: the filter architecture primarily manifests itself in the depth of the normalizing flow. Stacking numerous masked autoregressive flows in succession enables the model to address more intricate data distributions. 4) **Recursion depth**: setting a recursion depth that is excessively high aids in achieving a purer separation between the head and tail, a characteristic that is also evident in the results. However, it's important to note that an excessively high depth can substantially increase the time required for training. In contrast, when the depth is set to 2, we observe improved results compared to the baseline. Our analysis is closely tied to the characteristics of the sampled training data itself, where we observe feature overlap among different classes. Remarkably, the three recursive operations happen to align with the higher-density region of the classification boundary. 5) **Scalability**: the two-stage method we have introduced exhibits remarkable scalability. Whether applied in normalizing flow learning or in the subsequent classification of more balanced data, it can seamlessly integrate with existing techniques. The results unequivocally demonstrate that this stackable architecture significantly enhances the model's performance in the domain of long-tail intrusion detection. 6) **Extreme imbalance rate**: our method remains highly effective even in scenarios with an extremely high-class imbalance rate. When testing on the same dataset, we can still achieve results comparable to other methods even when the imbalance rate reaches 3000, indicating that the last class contains only one sample. Our method consistently performs well, achieving optimal results even when the class imbalance ratio is as high as 1000 or 3000, see Figure. 6.

## V. Conclusion

In our study, we propose to tackle the long-tailed issue in intrusion detection through the divide-then-conquer paradigm. First, we employ a Gaussian mixture flow filter to model the label space and generate clusters, which allows us to adjust the label space effectively and distinguish between head and tail samples. In the second stage, we perform separate detections for well-clustered samples and outliers. We believe that our findings not only improve our understanding of long-tailed intrusion detection but also offer valuable insights for future efforts in long-tailed semi-supervised learning in intrusion detection.

## Acknowledgments

This work was supported in part by the Supported by the Fundamental Research Funds for the Central Universities under Grant 2022RC026; in part by the CCF-Tecent Open Fund under Grant CCF-Tencent RAGR20220112; in part by the CCF-NSFOCUS Open Fund; in part by the NSFC Program under Grant 62202042, Grant 62076146, Grant 62021002, Grant U20A6003, Grant U19A2062, Grant 62127803, and Grant U1911401. The authors would like to express their gratitude to the anonymous reviewers for their valuable comments.

## References

[1] Hung-Jen Liao et al. "Intrusion detection system: A comprehensive review". In: *Journal of Network and Computer Applications* 36.1 (2013), pp. 16–24.

[2] Siyang Lu et al. "Black-box attacks against log anomaly detection with adversarial examples". In: *Information Sciences* 619 (2023), pp. 249–262.

[3] Siyang Lu et al. "SSDLog: a semi-supervised dual branch model for log anomaly detection". In: *World Wide Web* (2023), pp. 1–17.

[4] Nathan Shone et al. "A deep learning approach to network intrusion detection". In: *IEEE transactions on emerging topics in computational intelligence* 2.1 (2018), pp. 41–50.

[5] Hongyu Liu and Bo Lang. "Machine learning and deep learning methods for intrusion detection systems: A survey". In: *applied sciences* 9.20 (2019), p. 4396.

[6] Mikhail Belkin et al. "Reconciling modern machine-learning practice and the classical bias–variance trade-off". In: *Proceedings of the National Academy of Sciences* 116.32 (2019), pp. 15849–15854.

[7] Esteban G Tabak and Eric Vanden-Eijnden. "Density estimation by dual ascent of the log-likelihood". In: *Communications in Mathematical Sciences* 8.1 (2010), pp. 217–233.

[8] Esteban G Tabak and Cristina V Turner. "A family of nonparametric density estimation algorithms". In: *Communications on Pure and Applied Mathematics* 66.2 (2013), pp. 145–164.

[9] Laurent Dinh, David Krueger, and Yoshua Bengio. "Nice: Non-linear independent components estimation". In: *arXiv preprint arXiv:1410.8516* (2014).

[10] Juan P Agnelli et al. "Clustering and classification through normalizing flows in feature space". In: *Multiscale Modeling & Simulation* 8.5 (2010), pp. 1784–1802.

[11] Pavel Izmailov et al. "Semi-supervised learning with normalizing flows". In: *International Conference on Machine Learning*. PMLR. 2020, pp. 4615–4630.

[12] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. "Density estimation using real nvp". In: *arXiv preprint arXiv:1605.08803* (2016).

[13] Eric Nalisnick et al. "Hybrid models with deep and invertible features". In: *International Conference on Machine Learning*. PMLR. 2019, pp. 4723–4732.

[14] Yifan Zhang et al. "Deep long-tailed learning: A survey". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2023).

[15] Fannia Pacheco et al. "Towards the deployment of machine learning solutions in network traffic classification: A systematic survey". In: *IEEE Communications Surveys & Tutorials* 21.2 (2018), pp. 1988–2014.

[16] Ziwei Liu et al. "Large-scale long-tailed recognition in an open world". In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 2537–2546.

[17] Songyang Zhang et al. "Distribution alignment: A unified framework for long-tail visual recognition". In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2021, pp. 2361–2370.

[18] Bingyi Kang et al. "Decoupling representation and classifier for long-tailed recognition". In: *arXiv preprint arXiv:1910.09217* (2019).

[19] Ting Chen et al. "A simple framework for contrastive learning of visual representations". In: *International conference on machine learning*. PMLR. 2020, pp. 1597–1607.

[20] Bingyi Kang et al. "Exploring balanced feature spaces for representation learning". In: *International Conference on Learning Representations*. 2020.

[21] Yue Xu et al. "Constructing balance from imbalance for long-tailed image recognition". In: *European Conference on Computer Vision*. Springer. 2022, pp. 38–56.

[22] Sihao Yu et al. "A re-balancing strategy for class-imbalanced classification based on instance difficulty". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 70–79.

[23] Thomas Mensink et al. "Distance-based image classification: Generalizing to new classes at near-zero cost". In: *IEEE transactions on pattern analysis and machine intelligence* 35.11 (2013), pp. 2624–2637.

[24] Nour Moustafa and Jill Slay. "UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)". In: *2015 military communications and information systems conference (MilCIS)*. IEEE. 2015, pp. 1–6.

[25] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A Ghorbani. "Toward generating a new intrusion detection dataset and intrusion traffic characterization." In: *ICISSp* 1 (2018), pp. 108–116.

[26] Mahbod Tavallaee et al. "A detailed analysis of the KDD CUP 99 data set". In: *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*. 2009, pp. 1–6. DOI: 10.1109/CISDA.2009.5356528.

[27] Brian W Matthews. "Comparison of the predicted and observed secondary structure of T4 phage lysozyme". In: *Biochimica et Biophysica Acta (BBA)-Protein Structure* 405.2 (1975), pp. 442–451.

[28] Leon Danon et al. "Comparing community structure identification". In: *Journal of statistical mechanics: Theory and experiment* 2005.09 (2005), P09008.

[29] George Papamakarios, Theo Pavlakou, and Iain Murray. "Masked autoregressive flow for density estimation". In: *Advances in neural information processing systems* 30 (2017).

[30] Nitesh V Chawla et al. "SMOTE: synthetic minority over-sampling technique". In: *Journal of artificial intelligence research* 16 (2002), pp. 321–357.